

CQL 离线强化学习 风力发电机桨距角控制

——技术说明报告——

offline_pitch_rl 项目组

2026 年 6 月 20 日

目录

1	项目概述	2
1.1	问题背景	2
1.2	解决方案	2
2	系统架构	3
2.1	整体架构图	3
2.2	数据流	5
3	马尔可夫决策过程建模	6
3.1	状态空间	6
3.2	动作空间	6
3.3	奖励函数	6
3.4	转移与终止	7
4	数据流水线	8
4.1	数据加载	8
4.2	Z-score 归一化器	8
5	神经网络模型	9
5.1	Actor 策略网络	9
5.2	Critic Q 值网络	9
6	CQL 训练算法	11
6.1	训练循环总览	11
6.2	Critic 更新	11
6.2.1	Bellman 误差（标准 TD 学习）	11
6.2.2	CQL 保守性惩罚	11
6.2.3	Critic 总损失	11
6.3	Actor 更新：TD3+BC 自适应行为克隆	12
6.4	目标网络软更新	12
6.5	训练稳定性措施	12
7	关键设计决策与超参数	13
7.1	超参数总表	13
7.2	关键设计取舍	13
8	模型导出与部署	14
8.1	ONNX 导出	14
8.2	推理部署所需文件	14

9 可视化与评估体系	15
9.1 训练过程监控	15
9.2 策略评估	15
9.2.1 定量指标	15
9.2.2 评估结论等级	16
10 模块清单	17
A 运行命令	18
B 依赖环境	18
C CQL 损失函数公式总结	18

1 项目概述

1.1 问题背景

传统风力发电机桨距角控制依赖 PI 控制器。本项目的目标是利用离线历史数据训练一个强化学习策略网络 (Actor)，替代 PI 控制器输出桨距角指令。核心挑战在于：

1. 离线数据约束: 无法在线交互, 仅能使用 PI 控制器产生的历史数据 (PI_OfflineData.csv)
2. 分布外 (OOD) 动作过估计: 标准 Q-learning 会对数据分布外的动作产生过高的 Q 值估计
3. 策略安全约束: Actor 输出必须在物理可行范围 $[0, 1.57]$ rad 内, 且不能偏离数据分布太远

1.2 解决方案

采用 CQL (Conservative Q-Learning) 算法, 核心思路:

1. 保守 Q 值估计: 对 Critic 施加 log-sum-exp 正则化, 压低数据分布外随机动作的 Q 值
2. 行为克隆正则化: Actor 损失中加入 BC 项 $\text{MSE}(\pi(s), a_{\text{dataset}})$, 防止策略偏离数据分布
3. TD3+BC 自适应权重: $\lambda = \alpha / \text{mean}(|Q|)$ 动态平衡 Q 最大化与行为克隆
4. Z-score 归一化: 所有状态特征标准化, 提升训练数值稳定性
5. 奖励缩放: 将原始 $\sim [-200, 0]$ 奖励映射到 $\sim [-2, 0]$

2 系统架构

2.1 整体架构图

项目采用分层架构设计，自上而下分为配置层、数据层、模型层、训练引擎层、输出部署层和可视化层。

offline_pitch_rl 系统架构

config.py 配置文件 全局超参数 & 环境定义

data/ 数据层

- dataset.py
- CSV加载
 - 状态构造
 - 奖励计算
 - 五元组生成
- normalizer.py
- Z-score fit/transform/inverse
 - save/load 持久化

models/ 模型层

- actor.py
- 5→256→256→128→1
 - sigmoid 约束
- cql.py (Critic)
- (5+1)→256→256→128→1
 - Q(s, a) 值估计

train.py 训练引擎

- CQL Critic更新 (Bellman + logsumexp保守惩罚)
- Actor更新 (max Q + BC正则化 + 自适应)
- 目标网络软更新 (Polyak averaging)
- 梯度裁剪 & 训练历史记录

输出 & 部署层

actor_best.pth
(PyTorch模型)

actor.onnx
(ONNX部署)

training_
history.json

2.2 数据流

PI_OfflineData.csv

dataset.py

读取CSV

提取5维状态: [GenSpeed, SpeedError,
WindSpeed, PitchAngle, TowerAcc]

构造奖励: $-10 \cdot \text{err}^2 - 2 \cdot \text{tower_acc}^2$

缩放奖励: $\times 0.01$

构造next_state: shift(-1)

构造done: 最后一行为1

normalizer.fit(state)

state = normalizer.transform(state)

next_state = normalizer.transform(ns)

→ TensorDataset(s, a, r, ns, d)

DataLoader (batch_size=512, shuffle=True)

train.py 500 epochs output/

图 2: 数据流水线

3 马尔可夫决策过程建模

3.1 状态空间

状态向量 $\mathbf{s} \in \mathbb{R}^5$ ，五个维度如下：

表 1: 状态空间定义

维度	变量名	含义	单位	归一化
0	GenSpeed	发电机转速	–	Z-score
1	SpeedError	转速误差 ($\text{Ref} - \text{GenSpeed}$)	–	Z-score
2	WindSpeed	风速	–	Z-score
3	PitchAngle	当前桨距角测量值	–	Z-score
4	TowerAcc	塔架加速度	–	Z-score

状态维度 $\text{STATE_DIM} = 5$ ，所有状态经 Z-score 归一化（均值 0，标准差 1）。

3.2 动作空间

表 2: 动作空间定义

属性	值
维度	1 ($\text{ACTION_DIM} = 1$)
含义	桨距角指令 (Pitch Demand)
范围	$[\text{MIN_PITCH}, \text{MAX_PITCH}] = [0.0, 1.57] \text{ rad}$

Actor 输出通过 $\sigma(\cdot)$ (sigmoid) 映射到 $[0, 1]$ ，再缩放到 $[0, 1.57]$ ：

$$a = 0.0 + \sigma(f_{\theta}(\mathbf{s})) \times (1.57 - 0.0) \quad (1)$$

3.3 奖励函数

即时奖励由转速误差和塔架加速度两项惩罚组成：

$$r_{\text{raw}} = -10.0 \cdot (\text{speed_error})^2 - 2.0 \cdot (\text{tower_acc})^2 \quad (2)$$

$$r = r_{\text{raw}} \times 0.01 \quad (\text{缩放至 } \sim [-2, 0]) \quad (3)$$

设计意图：

- 转速误差惩罚（权重 10）：约束发电机转速跟随参考值
- 塔架加速度惩罚（权重 2）：抑制塔架振动，保护机械结构
- 奖励缩放（因子 0.01）：将原始范围 $\sim [-200, 0]$ 映射到 $\sim [-2, 0]$ ，避免大梯度破坏训练稳定性

3.4 转移与终止

- 下一状态 $\mathbf{s}_{t+1}$: 由于是离线数据, 采用 $\text{shift}(-1)$ 近似——即 $\mathbf{s}[t+1]$ 作为 $\mathbf{s}[t]$ 的下一状态, 最后一行的 $\mathbf{s}_{t+1}$ 设为自身
- 终止标志 d : 全零向量, 仅最后一行设为 1, 表示轨迹结束
- 折扣因子: $\gamma = 0.99$

4 数据流水线

4.1 数据加载

核心函数 `load_dataset(csv_file)` 位于 `data/dataset.py`，执行以下流程：

1. 使用 `pandas` 读取 CSV 文件
2. 提取五列原始数据：GenSpeed, GenSpeed_Ref, WindSpeed, PitchAngle_Mea, Tower-Acc
3. 构造转速误差： $\text{SpeedError} = \text{GenSpeed_Ref} - \text{GenSpeed}$
4. 构造动作：直接使用 `Action_PI_Dem` 列
5. 计算奖励： $r = (-10.0 \cdot e_{\text{speed}}^2 - 2.0 \cdot a_{\text{tower}}^2) \times 0.01$
6. 构造下一状态： $\text{next_state}[i] = \text{state}[i + 1]$, $\text{next_state}[-1] = \text{state}[-1]$
7. Z-score 归一化：`normalizer.fit(state) → normalizer.transform(state), normalizer.transform(...)`
8. 返回五元组 `(s, a, r, ns, d, norm)`

4.2 Z-score 归一化器

`data/normalizer.py` 中的 `Normalizer` 类实现 Z-score 标准化：

表 3: Normalizer 类方法

方法	功能
<code>fit(x)</code>	计算各维度均值 μ 和标准差 σ ；若 $\sigma < 10^{-8}$ 则置为 1.0（防零除）
<code>transform(x)</code>	$(x - \mu)/\sigma$
<code>inverse(x)</code>	$x \cdot \sigma + \mu$
<code>save(path)</code>	保存 μ/σ 到 JSON 文件
<code>load(path)</code>	从 JSON 加载 μ/σ （类方法）

归一化参数在训练时保存至 `output/normalization.json`，供推理部署时使用。

5 神经网络模型

5.1 Actor 策略网络

`models/actor.py` 定义策略网络 $\pi_{\theta}(\mathbf{s})$ ，结构如下：

输入: `state [B, 5]`

`Linear(5, 256) + ReLU`

`Linear(256, 256) + ReLU`

`Linear(256, 128) + ReLU`

`Linear(128, 1)`

`sigmoid` $\rightarrow [0, 1]$

`scale: MIN_PITCH + x * (MAX_PITCH - MIN_PITCH)`

输出: `pitch_demand [B, 1]` $[0, 1.57]$

设计要点:

- 三层隐藏层 (256 $\rightarrow$ 256 $\rightarrow$ 128)，使用 ReLU 激活函数
- 输出层不做激活，而是通过 sigmoid 缩放至合法动作范围
- 参数量约 101,633

5.2 Critic Q 值网络

`models/cql.py` 定义 Q 值网络 $Q_{\phi}(\mathbf{s}, \mathbf{a})$:

输入: `state [B, 5] + action [B, 1] = [B, 6]`

`concat(s, a) along dim=-1`

`Linear(6, 256) + ReLU`

`Linear(256, 256) + ReLU`

`Linear(256, 128) + ReLU`

`Linear(128, 1)`

输出: `Q(s, a) [B, 1]`

设计要点:

- 状态与动作在最后一维拼接后送入网络
- 与 Actor 结构对称（隐藏层维度一致），便于训练平衡
- 目标网络 $\bar{Q}_{\phi'}$ 为 Critic 的 Polyak 平均副本

6 CQL 训练算法

6.1 训练循环总览

训练主循环位于 `train.py`, 共 500 个 epoch, 每个 batch 执行三步更新:

1. **Critic 更新**: 最小化 Bellman 误差 + CQL 保守性惩罚
2. **Actor 更新**: 最大化 Q 值 + 行为克隆正则化 (TD3+BC 自适应 λ)
3. **目标网络软更新**: Polyak 平均

6.2 Critic 更新

6.2.1 Bellman 误差 (标准 TD 学习)

$$\tilde{a} = \pi(\mathbf{s}_{t+1}) \quad (\text{目标策略动作})$$

$$\tilde{q} = \bar{Q}(\mathbf{s}_{t+1}, \tilde{a}) \quad (\text{目标 Q 值})$$

$$y = r + \gamma \cdot (1 - d) \cdot \tilde{q} \quad (\text{TD 目标})$$

$$\mathcal{L}_{\text{bellman}} = \frac{1}{B} \sum_{i=1}^B (Q(\mathbf{s}_i, a_i) - y_i)^2 \quad (\text{Bellman MSE})$$

6.2.2 CQL 保守性惩罚

对每批次数据采样 $N = 10$ 个均匀随机动作, 在动作维度上计算 log-sum-exp:

$$a_{\text{rand}} \sim \mathcal{U}(0, \text{MAX_PITCH})^{B \times N \times 1} \quad (\text{随机动作})$$

$$Q_{\text{rand}} = Q(\mathbf{s}, a_{\text{rand}}) \in \mathbb{R}^{B \times N} \quad (\text{随机动作 Q 值})$$

$$\mathcal{L}_{\text{cql}} = \frac{1}{B} \sum_{i=1}^B \log \sum_{j=1}^N \exp Q_{\text{rand}}[i, j] - \frac{1}{B} \sum_{i=1}^B Q(\mathbf{s}_i, a_i) \quad (\text{CQL 惩罚})$$

直觉解释:

- log-sum-exp 项推高所有随机动作的 Q 值 $\rightarrow$ 优化时该项降低 $\rightarrow$ 随机动作 Q 值被压低
- $-\text{mean}(Q)$ 项确保数据集中真实动作的 Q 值不被压低

6.2.3 Critic 总损失

$$\mathcal{L}_{\text{critic}} = \mathcal{L}_{\text{bellman}} + \alpha_{\text{cql}} \cdot \mathcal{L}_{\text{cql}} \quad (4)$$

其中 $\alpha_{\text{cql}} = 1.0$ 控制保守性强度。

6.3 Actor 更新: TD3+BC 自适应行为克隆

$$q_{\text{pred}} = Q(\mathbf{s}, \pi(\mathbf{s})) \quad (\text{当前策略 } Q \text{ 值})$$

$$\mathcal{L}_{\text{bc}} = \frac{1}{B} \sum_{i=1}^B (\pi(\mathbf{s}_i) - a_i)^2 \quad (\text{行为克隆损失})$$

$$\lambda = \frac{\lambda_{\text{bc}}}{\text{mean}(|Q|) + 10^{-6}} \quad (\text{自适应权重})$$

$$\mathcal{L}_{\text{actor}} = -\lambda \cdot \frac{1}{B} \sum_{i=1}^B q_{\text{pred},i} + \mathcal{L}_{\text{bc}} \quad (\text{Actor 总损失})$$

自适应 λ 机制: 当 Q 值幅度大时自动降低 BC 权重, 当 Q 值幅度小时增大 BC 权重——保证 BC 与 Q 最大化的稳健平衡。

6.4 目标网络软更新

$$\phi' \leftarrow \tau \cdot \phi + (1 - \tau) \cdot \phi', \quad \tau = 0.005 \quad (5)$$

Polyak 平均使目标网络平滑变化, 稳定 TD 学习目标。

6.5 训练稳定性措施

表 4: 训练稳定性措施汇总

措施	参数/方式	目的
梯度裁剪	GRAD_CLIP = 10.0	防止梯度爆炸
奖励缩放	REWARD_SCALE = 0.01	数值稳定
状态归一化	Z-score	消除量纲差异
降低保守性	CQL_ALPHA = 1.0 (原 5.0)	避免 Q 值过度悲观
学习率一致	LR_CRITIC = 1e-4	训练平衡

7 关键设计决策与超参数

7.1 超参数总表

表 5: 全局超参数配置

参数	值	说明
STATE_DIM	5	状态维度
ACTION_DIM	1	动作维度
MIN_PITCH	0.0	最小桨距角 (rad)
MAX_PITCH	1.57	最大桨距角 (rad)
BATCH_SIZE	512	批大小
GAMMA	0.99	折扣因子
TAU	0.005	目标网络软更新系数
LR_ACTOR	10^{-4}	Actor 学习率
LR_CRITIC	10^{-4}	Critic 学习率
CQL_ALPHA	1.0	CQL 保守性惩罚权重
CQL_NUM_SAMPLES	10	每状态随机动作采样数
LAMBDA_BC	2.5	BC 正则化基础权重
REWARD_SCALE	0.01	奖励缩放因子
GRAD_CLIP	10.0	梯度裁剪阈值
EPOCHS	500	训练轮数
DEVICE	cuda	计算设备

7.2 关键设计取舍

表 6: 设计决策理由分析

设计选择	理由
CQL over BC-Q/BEAR	CQL 实现简单, log-sum-exp 惩罚在动作维度上直接且有效
TD3+BC 自适应 λ over 固定 λ	Q 值幅度随训练变化, 固定权重难以平衡 Q 最大化与 BC 正则化
Z-score over MinMax 归一化	Z-score 对异常值更鲁棒, 保持分布形状
单 Critic over 双 Critic	简化实现; CQL 本身的保守性已提供足够的低估偏差
ONNX opset 11	广泛兼容的部署标准
批大小 512 over 256	离线数据充足时, 大批量提供更稳定的梯度估计

8 模型导出与部署

8.1 ONNX 导出

export_onnx.py 将训练好的 Actor 模型导出为 ONNX 格式：

```
1 model = Actor(STATE_DIM)
2 model.load_state_dict(torch.load("output/actor_best.pth", map_location="
   cpu"))
3 model.eval()
4
5 dummy = torch.randn(1, STATE_DIM)
6
7 torch.onnx.export(
8     model, dummy, "output/actor.onnx",
9     input_names=["state"],
10    output_names=["pitch_demand"],
11    dynamic_axes={
12        "state": {0: "batch"},
13        "pitch_demand": {0: "batch"}
14    },
15    opset_version=11
16 )
```

Listing 1: ONNX 导出流程

导出配置：

- input_names: ["state"]
- output_names: ["pitch_demand"]
- dynamic_axes: batch 维度设为动态，支持任意批次大小
- opset_version: 11

8.2 推理部署所需文件

表 7: 部署文件清单

文件	用途
output/actor.onnx	ONNX 模型（跨平台推理）
output/actor_best.pth	PyTorch 权重（可继续训练）
output/normalization.json	归一化参数（推理时需对输入做相同归一化）

9 可视化与评估体系

9.1 训练过程监控

`visualization/plot_training.py` 生成三类图表，所有曲线包含原始值和 10-epoch 滑动平均平滑线，支持中英双语标注。

表 8: 训练监控图表

图表	内容	输出文件
Loss Curves	Actor/Critic/Bellman/CQL 四合一损失曲线	training_losses.png
Q-Value & Return	平均 Q 值趋势 & 平均即时奖励	q_value_return.png
Training Summary	6 合 1 综合训练摘要图	training_summary.png

9.2 策略评估

`visualization/evaluate.py` 提供四维度策略质量评估：

表 9: 评估分析维度

评估维度	分析方法	输出文件
动作对比	散点图 (pred vs true)、误差直方图、时序对比、分布对比	action_comparison.png
策略质量	动作偏差分布、逐样本偏差、状态-动作关系	policy_quality.png
状态覆盖	5 维状态 vs 动作散点图、动作值分布	state_action_coverage.png

9.2.1 定量指标

评估输出以下指标：

- MAE (Mean Absolute Error): $\frac{1}{N} \sum |\pi(\mathbf{s}_i) - a_i|$
- RMSE (Root Mean Square Error): $\sqrt{\frac{1}{N} \sum (\pi(\mathbf{s}_i) - a_i)^2}$
- Max Absolute Error: $\max_i |\pi(\mathbf{s}_i) - a_i|$
- 动作均值/标准差对比
- 奖励统计

9.2.2 评估结论等级

表 10: MAE 等级判定

MAE 范围	等级
< 0.01	优秀 (EXCELLENT) — Actor 紧密匹配行为策略
< 0.05	良好 (GOOD) — 与行为策略有轻微偏差
< 0.10	一般 (FAIR) — 明显偏差, 需检查训练
≥ 0.10	较差 (POOR) — 偏差较大, 训练可能需要调参

10 模块清单

表 11: 项目模块一览

文件路径	职责	行数
<code>config.py</code>	全局超参数配置	29
<code>train.py</code>	CQL 训练主循环	241
<code>export_onnx.py</code>	ONNX 模型导出	47
<code>data/dataset.py</code>	CSV 加载、状态/奖励构造、五元组生成	83
<code>data/normalizer.py</code>	Z-score 归一化器（含持久化）	46
<code>models/actor.py</code>	Actor 策略网络 π_θ	40
<code>models/cql.py</code>	Critic Q 值网络 Q_ϕ	33
<code>env/reward.py</code>	奖励函数占位（实际在 <code>dataset.py</code> ）	33
<code>utils/save_scaler.py</code>	归一化器保存占位（已集成至 <code>Normalizer</code> ）	2
<code>utils/load_scaler.py</code>	归一化器加载占位（已集成至 <code>Normalizer</code> ）	2
<code>visualization/plot_training.py</code>	训练曲线可视化（中英双语）	208
<code>visualization/evaluate.py</code>	策略评估与可视化（中英双语）	268

A 运行命令

```

1 # 1. 训练
2 python train.py
3
4 # 2. 生成训练曲线
5 python visualization/plot_training.py
6
7 # 3. 策略评估
8 python visualization/evaluate.py
9
10 # 4. ONNX 导出
11 python export_onnx.py

```

Listing 2: 运行命令

B 依赖环境

项目所需 Python 包:

torch, numpy, pandas, matplotlib, onnx

C CQL 损失函数公式总结

Critic 总损失

$$\mathcal{L}_{\text{critic}} = \underbrace{\mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\left(Q(s,a) - (r + \gamma(1-d)\bar{Q}(s',\pi(s'))) \right)^2 \right]}_{\text{Bellman TD Error}} + \alpha \cdot \underbrace{\left[\mathbb{E}_{s \sim \mathcal{D}} \left[\log \sum_{a'} \exp Q(s,a') \right] - \mathbb{E}_{(s,a) \sim \mathcal{D}} [Q] \right]}_{\text{CQL Conservative Penalty}} \quad (6)$$

Actor 总损失

$$\mathcal{L}_{\text{actor}} = - \underbrace{\frac{\lambda_{\text{bc}}}{\text{mean}(|Q|)}}_{\text{自适应}\lambda} \cdot \mathbb{E}_{s \sim \mathcal{D}} [Q(s, \pi(s))] + \underbrace{\mathbb{E}_{(s,a) \sim \mathcal{D}} [(\pi(s) - a)^2]}_{\text{BC Regularization}} \quad (7)$$

目标网络更新

$$\phi' \leftarrow \tau \cdot \phi + (1 - \tau) \cdot \phi' \quad (8)$$

——文档结束——